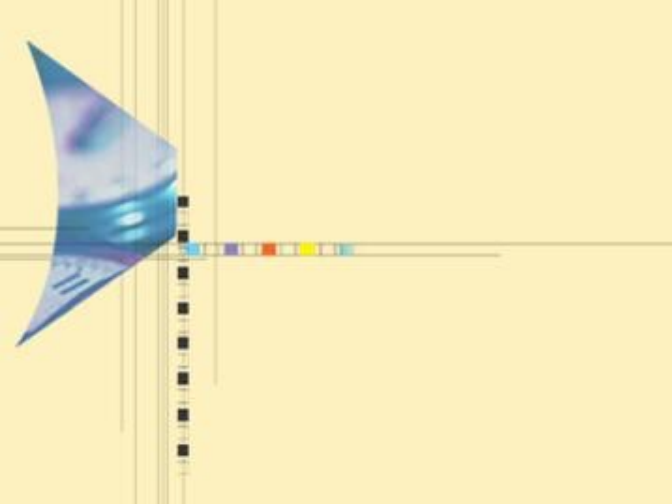




算法设计与分析

主讲：刘娟，武汉大学人工智能学院

2024-2025 第二学期，1-16周

周一(3-4，三区1-502)，周三（6-7，三区1-325）

第十一讲

近似算法





主要内容

11.1 引言

11.2 基本定义

11.3 近似算法分析

11.4 可近似问题算法举例

11.5 步可近似问题举例

11.6 完全可近似问题举例

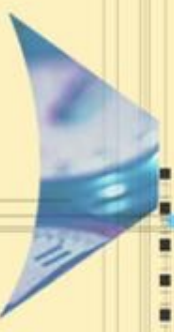


11.1 引言

- 许多困难的组合优化问题无法通过回溯或随机化有效解决。
- 在这种情况下，解决其中一些问题的另一种方法是设计一种近似算法，前提是我们满足于接近最优解的“合理”解。
- 每种近似算法都有一个性能界来保证对一个给定实例的解不会远离精确解的邻域。
- 大多数近似算法的一个显著特征是很快，因为它们大多是基于贪心的启发式方法。
- 但是，贪心算法的正确性证明可能比较复杂。一般来讲，近似算法性能界越好，证明这个算法的正确性越难。

11.1 引言

- 近似算法是用于寻找优化问题近似解的算法。
- 近似算法通常与NP难问题的求解相关联。由于不太可能有多项式时间的精确算法来解决NP难问题，因此只能采用多项式时间次优解。但有些NP难问题的“合理”近似算法也可能不存在，除非有 $NP=P$ 。
- 与通常只快速找到相当好的解的启发式方法不同，在近似算法要可证明的解的质量和可证明的运行时间的界。



主要内容

11.2 基本定义

11.2.1 组合优化问题

11.2.2 近似算法

11.2.3 差界

11.2.4 近似度（近似比）

11.2.5 多项式近似方案

11.2.6 完全多项式近似方案

11.2.7 伪多项式时间算法

11.2.1 组合优化问题

组合优化问题 Π 不是最小化问题就是最大化问题。它由三个部分组成：

- (1) 一个实例集合 D_{Π} ；
- (2) 对于每个实例 $I \in D_{\Pi}$, 存在 I 的一个候选解的有限集合 $S_{\Pi}(I)$ ；
- (3) 实例 $I \in D_{\Pi}$, 的每个解 $\sigma \in S_{\Pi}(I)$, 存在一个值 $f_{\Pi}(\sigma)$, 称为 σ 的解值。

最优解：如果 Π 是最小化问题，那么实例 $I \in D_{\Pi}$ 的最优解 σ^* 有如下属性：对于所有的 $\sigma \in S_{\Pi}(I)$, $f_{\Pi}(\sigma^*) \leq f_{\Pi}(\sigma)$ 。最大化问题的最优解也类似定义。最优的解值 $f_{\Pi}(\sigma^*)$ 记为 $\text{OPT}(I)$ 。

11.2.2 近似算法

近似算法：最优化问题 Π 的一个近似算法 A 是一个（多项式时间）算法，给定一个实例 $I \in D_{\Pi}$ ，算法输出某个解 $\sigma \in S_{\Pi}(I)$ 。算法 A 得到的解也称为近似解，值 $f_{\Pi}(\sigma)$ 记为 $A(I)$ 。

近似算法的时间复杂性必须是多项式的。这是近似算法的基本目标。

例 1

- 装箱问题 (BIN PACKING) : 给定一个大小在0和1之间的物品的集合, 要求把这些物品装到容量大小为1的箱子中, 要求箱子数目最少。(显然是最小化问题)
- 实例集 D_{Π} 由所有实例 $I = \{s_1, s_2, \dots, s_n\}$ 组成, 使得对于所有 j ($1 \leq j \leq n$), s_j 在0到1之间。
- 解集合 $S_{\Pi}(I)$ 由子集的集合 $\sigma = \{B_1, B_2, \dots, B_k\}$ 组成, 它是 I 中的不相交划分, 是的对于所有的 j ($1 \leq j \leq k$), 有

$$\sum_{s \in B_j} s \leq 1$$

- 给出一个解 σ , 它的值 $f_{\Pi}(\sigma)$ 是 $|\sigma| = k$

例 1

- 这个问题的最优解是符合条件的具有最小子集数的 σ .
- 令 A 是为每一物品分配一个箱子的 (平凡) 算法。则由于定义可知, A 是一个近似算法。
- 显然, A 不是一个好的近似算法。

11.2.3 差界

- 也许，对近似算法的最大希望是，最优解的值与近似算法获得的解的值之间的差始终是恒定的。
- 换句话说，对于问题的所有实例 I ，最希望由近似算法 A 得到的解是使得 $|A(I)-OPT(I)| \leq K$ ，其中 K 是常数。

定义：如果能证明对每个实例 I 都有下式成立，则称近似算法 A 具有有界误差或差界 K （ K 为常数）：

$$OPT(I)-K \leq A(I) \leq OPT(I)+K, \text{ 也即 } |A(I)-OPT(I)| \leq K$$

也称为近似算法的绝对性能保证（absolute performance guarantee）

- 但是，只有很少的NP困难的最优化问题的差界是已知的。

11.2.4 相对性能界

- 定义：设A 是求解优化问题 Π 的近似算法，I 是 Π 的一个问题实例. 解A(I)的近似度（近似比） $R_A(I)$ 定义为：

$$R_A(I) = \max\left(\frac{A(I)}{OPT(I)}, \frac{OPT(I)}{A(I)}\right)$$

- 显然，近似度（近似比）大于等于1。
- 有了近似度（近似比）的定义，就可以对算法A产生的解的质量有了统一的测度。与具体实例有关的近似度也称相对近似度。

11.2.4 相对性能界

定义：近似算法A的绝对近似度 R_A 定义如下：

$$R_A = \inf\{r \geq 1 \mid R_A(I) \leq r, \text{ 对所有实例 } I \in D_\Pi\}$$

- 也就是说， R_A 是在所有可能的问题实例上的最大的近似度。

定义：近似算法的渐进近似度 R_A^∞ 定义如下：

$$R_A^\infty = \inf\{r \geq 1 \mid \text{对某个整数 } N, R_A(I) \leq r, \text{ 对所有实例 } I \in D_\Pi, \\ |I| \geq N\}$$

- 也就是说， R_A^∞ 是在所有大小不小于N的问题实例的绝对近似度。
- 对于一些问题渐进近似度比绝对近似度更合适，而对另一些问题，两种近似度相同。

11.2.5 多项式近似方案

定义： 一个最优化问题的近似方案(approximation scheme)是一个算法簇 $\{A_\varepsilon \mid \varepsilon > 0\}$ ，使得 $R_{A_\varepsilon} \leq 1 + \varepsilon$.

- 近似方案是近似度收敛于1的一系列算法。
- 将一个近似方案看成是一个近似算法 A ，它的输入问题的一个实例 I 和有界误差 ε ，使得 $R_A(I, \varepsilon) \leq 1 + \varepsilon$

定义： 一个多项式近似方案(polynomial approximation scheme, PAS) 是一个近似方案 $\{A_\varepsilon \mid \varepsilon > 0\}$ ，其中每个算法 A_ε 在输入实例 I 的大小的多项式时间内运行。

- 注意：根据PAS的定义， A_ε 的运行时间相对于输入实例 I 的大小是多项式的，但相对于 $1/\varepsilon > 0$ 可能不是多项式的。

11.2.6 完全多项式近似方案

定义：一个完全多项式近似方案 (fully polynomial approximation scheme, FPAS) 是一个近似方案 $\{A_\epsilon\}$ ，其中每个近似算法 A_ϵ 在以输入实例的大小和 $1/\epsilon$ 两者的多项式时间内运行。

- PAS的运行时间对于 $1/\epsilon$ 可能是以指数形式增长的， $1/\epsilon$ 是预期的误差界的倒数。而FPAS的运行时间同时也是 $1/\epsilon$ 的多项式时间的。

11.2.7 伪多项式时间算法

定义：伪多项式时间算法是一类算法，它们在 L 值的多项式时间内运行，其中 L 是输入实例的最大数。

- 例如求解0/1背包问题的动态规划算法的运行时间为 $O(nC)$ ， C 是背包容量。该算法是伪多项式时间算法。
- 注：如果算法在 $\log L$ 的多项式时间内运行，则它是一个多项式时间算法。
- 对于存在伪多项式时间算法 A 的NP困难问题，找一个FPAS的一般思路：从 A 开始，对于实例 I 的输入值应用标度变换和舍入得到实例 I' ，将算法 A 应用到修改得到的实例 I' 上得出一个答案，它就是相对于实例 I 的最优解的近似解。



主要内容

11.1 引言

11.2 基本定义

11.3 近似算法分析

11.4 可近似问题算法举例

11.5 不可近似问题举例

11.6 完全可近似问题举例



11.3 近似算法的分析

近似算法的分析：算法的运行时间与近似比.

分析近似比

- (1) 估计最优解的值(建立最优值与近似解的值之间的关系)
- (2) 构造使算法产生最坏的解的实例. 如果这个解的值与最优值的比达到或者可以任意接近得到的近似比(这样的实例称作**紧实例**), 那么说明这个近似比已经是最好的、不可改进的了; 否则说明还有进一步的研究余地。

研究问题本身的可近似性

即在 $P \neq NP$ (或其他更强)的假设下, 该问题近似算法的近似比的下界.

可近似性分类

假设 $P \neq NP$, NP难的组合优化问题按可近似性可分成三类:

- (1) **完全可近似的**: 对任意小的 $\varepsilon > 0$, 存在 $(1+\varepsilon)$ -近似算法
0/1背包问题
- (2) **可近似的**: 存在具有常数比的近似算法
最小顶点覆盖问题、多机调度问题
- (3) **不可近似的**: 不存在具有常数比的近似算法
货郎问题



主要内容

11.1 引言

11.2 基本定义

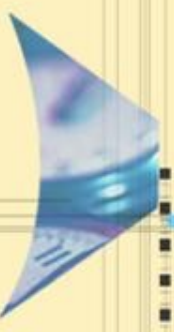
11.3 近似算法分析

11.4 可近似问题算法举例

11.5 不可近似问题举例

11.6 完全可近似问题举例





主要内容

11.4 可近似问题举例

11.4.1 装箱问题

11.4.2 ETSP问题

11.4.3 最小顶点覆盖问题

11.4.4 多机调度问题



11.4.1 装箱问题

四种启发式方法：

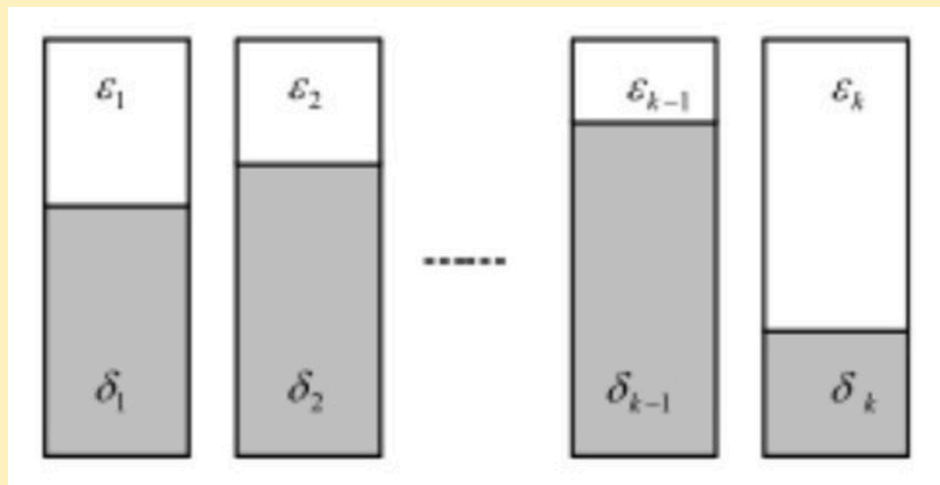
- 最先适配 (FF)：箱子被编号为 $1, 2, \dots$ 所有箱子最初都是空的。物品被考虑按 $u_1, u_2, \dots, u_n$ 的顺序装入箱。为了装入物品 u_i ，找出最小序号，使得箱子 j 最多装了 $1-s_i$ ，并将物品 u_i 到箱子 j 中。（分配第一个能装的箱子）
- 最佳适配 (BF)：此方法与FF方法相同，除了要装物品 u_i 时，查找满足已填充水平 $l \leq 1-s_i$ ，并且 l 最大的箱子。（分配能装的箱子中剩余空间最小的）
- 递减最先适配 (FFD)：物品首先按大小降序排列，然后使用FF方法装箱。
- 递减最佳适配 (BFD)：物品首先按大小降序排列，然后使用BF方法装箱。

FF的近似度分析

- 假定：箱子容量为1个单位体积，物品大小 $s_i \leq 1$ ， $i=1,2,\dots,n$
- 令 $FF(I)$ 表示在实例 I 下，使用算法FF装入物品时，所使用的箱子数目；
- 令 $OPT(I)$ 表示最优装入时所使用的箱子数目。
- 根据FF装箱规则，至多有一个非空的箱子所装的物品体积小于 $1/2$ 。
 - 如果有两个或以上的箱子所装的物品体积小于 $1/2$ 。假设有两个这样的箱子 b_i ， b_j ，并且 $i < j$ 。
 - 由于 b_i ， b_j 中物品的体积均小于 $1/2$ ，根据算法的装入规则，必然会把 b_j 中的物品继续装入 b_i ，而不会装入另外箱子。

FF的近似度分析

- 因此，根据FF规则，装入箱子的物品如下图，其中 ε_i 为箱子中的空余体积， δ_i 为箱子中装入的物品体积：



- 因此有

$$\sum_{i=1}^k \delta_i = \sum_{i=1}^n s_i \quad \text{及} \quad \varepsilon_i < \delta_i, \quad i=1, 2, \dots, k-1$$

- 对于第k个箱子，或者 $\varepsilon_k < \delta_k$ ，或者 $\varepsilon_k > \delta_k$ ，

FF的近似度分析

- 对于前者, 显然有

$$\sum_{i=1}^k \varepsilon_i < \sum_{i=1}^k \delta_i = \sum_{i=1}^n s_i$$

- 对于后者, 则有 $\varepsilon_{k-1} < \delta_k$, $\varepsilon_k < \delta_{k-1}$, 因此有 $\varepsilon_{k-1} + \varepsilon_k < \delta_{k-1} + \delta_k$, 因此, 也有

$$\sum_{i=1}^k \varepsilon_i < \sum_{i=1}^k \delta_i = \sum_{i=1}^n s_i$$

- 因此, 无论是哪种情况, 总有

$$\sum_{i=1}^k \varepsilon_i < \sum_{i=1}^k \delta_i = \sum_{i=1}^n s_i$$

- 所以,

$$FF(I) = \sum_{i=1}^k \delta_i + \sum_{i=1}^k \varepsilon_i < 2 \sum_{i=1}^n s_i$$

FF的近似度分析

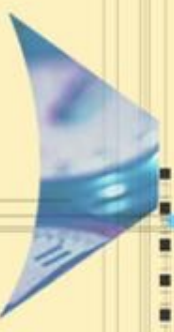
- 在最优化装入时，所有装物品的箱子全部装满，即：

$$OPT(I) = \sum_{i=1}^n s_i$$

- 因此，FF算法的近似比为：

$$\rho_{FF}(I) = \frac{FF(I)}{OPT(I)} < 2$$

- 即，在任意实例I上，FF算法的近似比小于2。



实际上，可进一步证明：

- 定理 11.1 对于装箱问题的所有实例 I ,

$$FF(I) \leq \frac{17}{10} OPT(I) + 2$$

可以证明，BF 方法也有一个 $17/10$ 的近似度。

- 定理 11.2 对于装箱问题的所有实例 I ,

$$FFD(I) \leq \frac{11}{9} OPT(I) + 4$$

同样可以证明 BFD 方法也有一个 $11/9$ 的近似度。



11.4.2 欧几里得旅行商问题 (Euclidean TSP)

问题描述:

给定一个平面上 n 个点的集合 S , 找出一个在这些点上的最短长度的游程 τ 。这里一个游程是恰好访问每个点一次的一条回路。

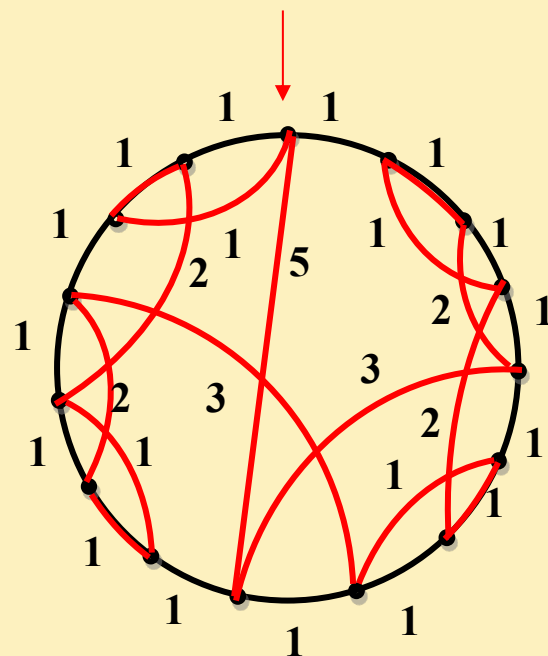
ETSP问题是 TSP问题的特殊情况, 点之间的距离是欧式距离, 满足三角不等关系。

最近邻 (NN) 法

基本思想：采用贪心法。设 p_1 是任意一个起始点，下一个访问距离 p_1 最近的点，比如 p_2 ，然后再访问距离 p_2 最近的点，等等。

一个使得NN性能很坏的实例 I ,

$$NN(I)=27, OPT(I)=15$$



NN法的性能分析

定理 对于ETSP问题所有满足三角不等式的 n 个城市的实例 I , 总有

$$\text{NN}(I) \leq \frac{1}{2} (\lceil \log_2 n \rceil + 1) \text{OPT}(I).$$

而且, 对于每一个充分大的 n , 存在满足三角不等式的 n 个城市的实例 I 使得

$$\text{NN}(I) > \frac{1}{3} \left(\log_2(n+1) + \frac{4}{3} \right) \text{OPT}(I).$$

NN法的性能分析

- 根据前面定理，对于大小为 n 的输入实例 I ，NN的近似比：

$$R_{NN}(I) = \frac{NN(I)}{OPT(I)} = O(\log n).$$

显然， $R_{NN} = \infty$ 。

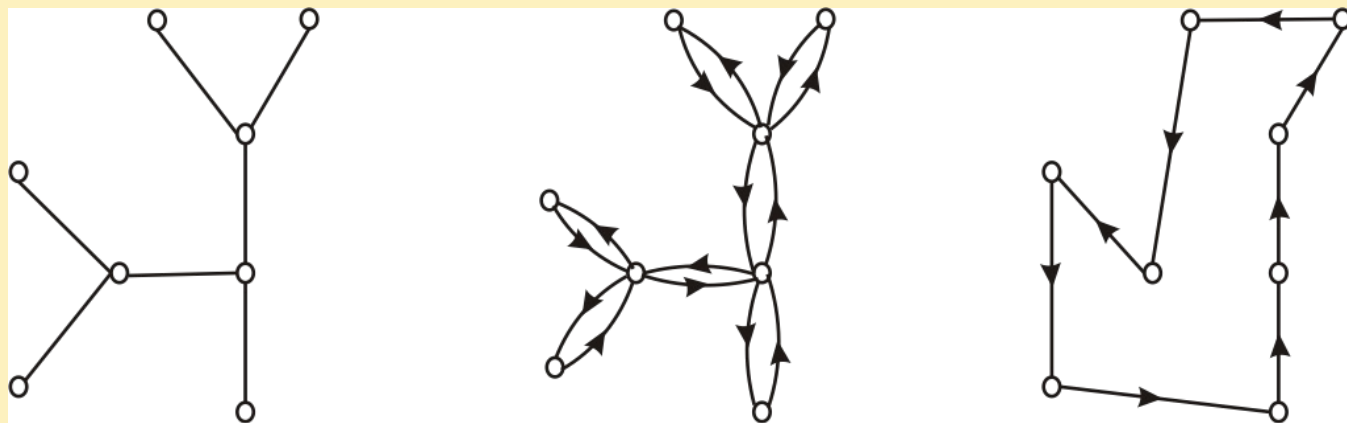
结论：NN算法不是常数近似比的算法，不能实际应用。

对于ETSP问题，是否存在常数近似比的算法？

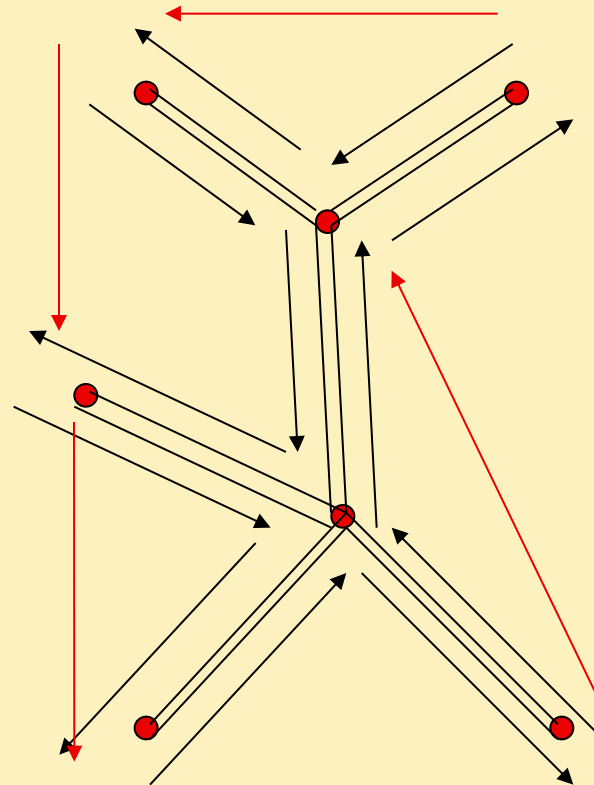
最小生成树 (MST) 方法

首先, 求图的一棵最小生成树 T . 然后, 沿着 T 走两遍得到图的一条欧拉回路. 最后, 顺着这条欧拉回路, 跳过已走过的顶点, 抄近路得到一条哈密顿回路。

例



- 求最小生成树和欧拉回路都可以在多项式时间内完成, 故算法是多项式时间的.
- 由于满足三角不等式, 抄近路不会增加长度



EMST Eulerian Graph Hamiltonian tour

MST方法性能分析

定理 对欧式货郎问题的任何实例 I , 均有

$$\text{MST}(I) < 2\text{OPT}(I)$$

证明:

- 从哈密顿回路中删去一条边就得到一棵生成树, 故 T 的权小于 $\text{OPT}(I)$;
- 沿 T 走两遍的长小于 $2\text{OPT}(I)$;
- 因为满足三角不等式, 抄近路不会增加长度, 故 $\text{MST}(I) < 2\text{OPT}(I)$.
- MST是2-近似算法.

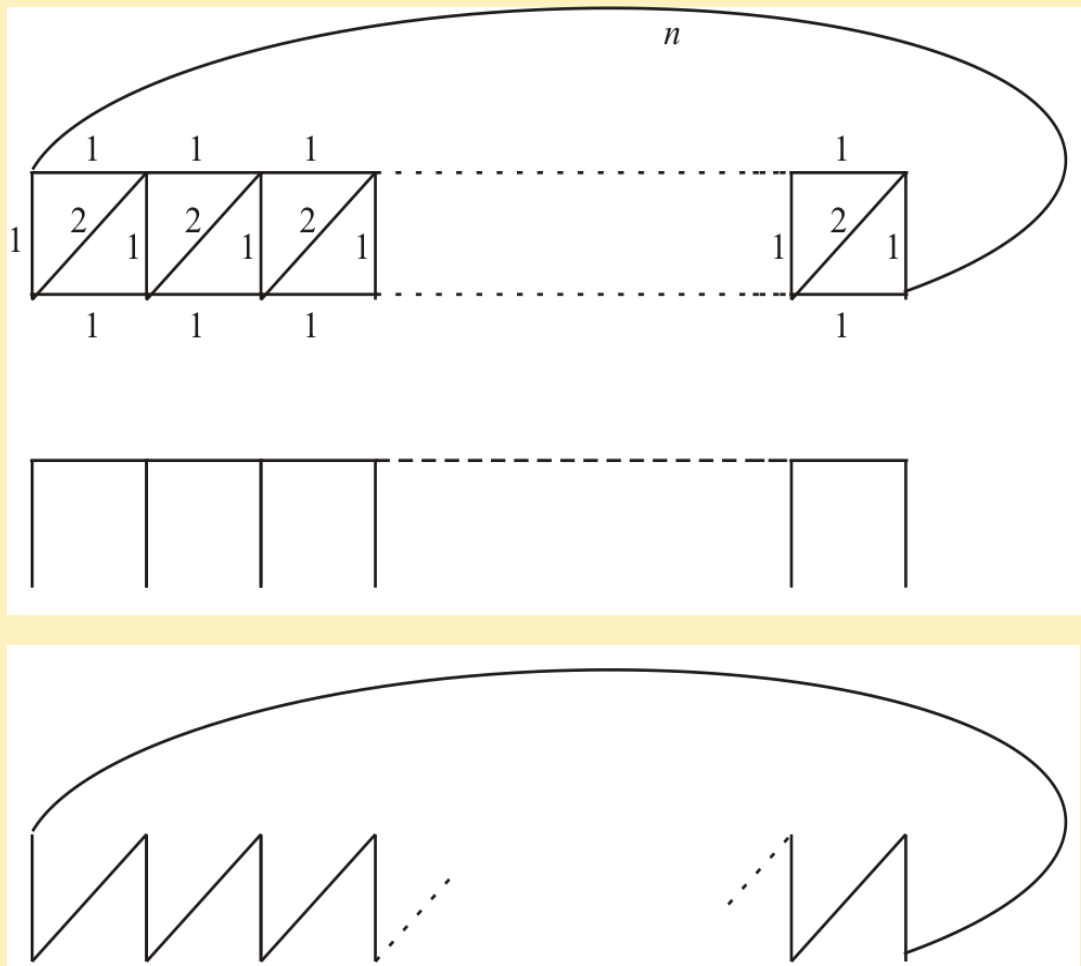
紧实例

2n个城市

$$\text{OPT}(I) = 2n$$

$$\text{MST}(I) = n + 2(n-1) + n = 4n - 2$$

$$= \left(2 - \frac{1}{n}\right) \text{OPT}(I)$$

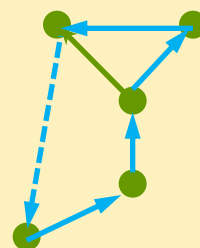
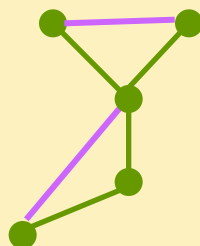
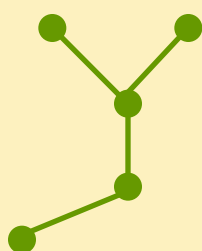


结论：MST给出的近似比结果已经是最好的

最小权匹配 (MM) 法

首先求图的一棵最小生成树 T 。

记 T 的所有奇度顶点集为 X , X 一定有偶数个顶点 (图的性质), 在原图中求 X 的最小匹配 M ; 把 M 加入 T 得到一个欧拉图, 求这个欧拉图的欧拉回路; 沿着这条欧拉回路, 跳过已走过的顶点, 抄近路得到一条哈密顿回路。



求任意图最小权匹配的算法是多项式时间的, 因此 MM 是多项式时间的。

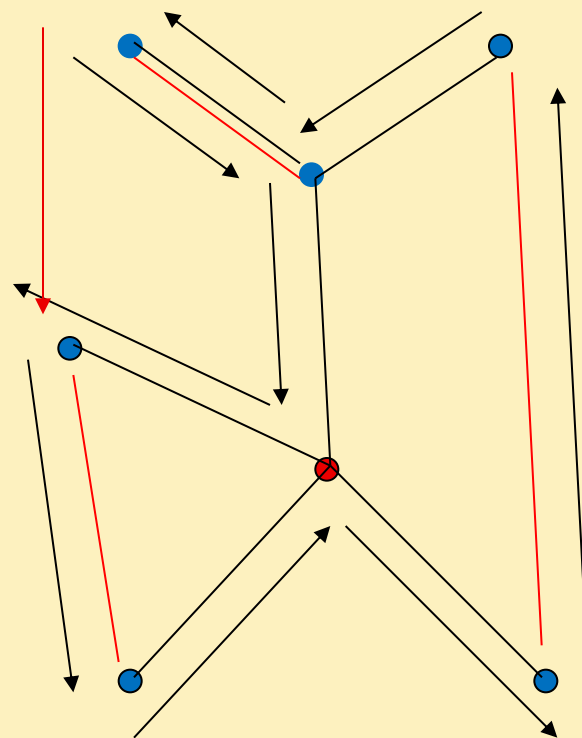
最小权匹配 (MM) 法

算法 11.1 ESTPAPPROX

Input: ETSP问题的一个实例 I .

Output: 实例 I 的一个游程 τ .

1. 找出 I 的最小生成树 T .
2. 识别 T 中度为奇数的顶点集合 X .
3. 求 X 中顶点的最小权重匹配 M .
4. 在 $T \cup M$ 中查找欧拉游程 τ_e .
5. 按边跟踪 τ_e , 跳过已走过的顶点, 抄近路得到哈密顿回路 τ .



EMST Eulerian Graph Hamiltonian tour

MM法的性能分析

定理 对ETSP问题的所有满足三角不等式的实例 I ,

$$\text{MM}(I) < \frac{3}{2} \text{OPT}(I).$$

MM 是 $3/2$ -近似算法.

证明:

- 从哈密顿回路中删去一条边就得到一棵生成树, 故:
 $\text{Length}(T) < \text{OPT}(I)$;
- M 是 X 的最小匹配, 故 $\text{Length}(M) \leq 1/2 \text{OPT}(I)$;
- $\text{MM}(I) = \text{length}(T) + \text{length}(M)$
 $< \text{OPT}(I) + (1/2)\text{OPT}(I)$
 $= (3/2)\text{OPT}(I)$

11.4.3 最小顶点覆盖问题

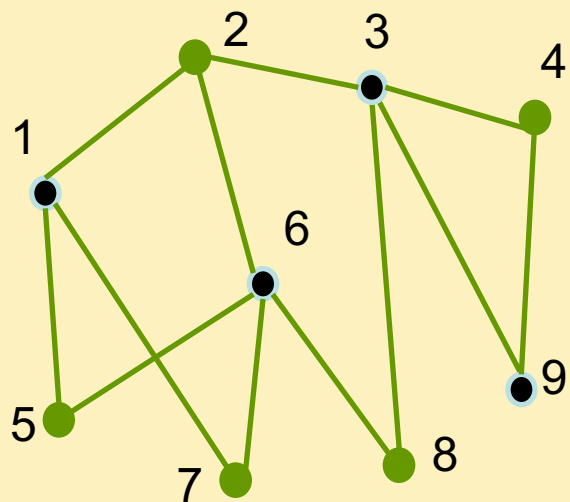
问题描述:

给定一个图 $G=(V,E)$ ，找最小的顶点覆盖集合。

- 直观的启发式方法：
 - 重复执行以下步骤直到 E 变为空：(1) 任意选取一条边 e ，并将其端点之一（如 v ）添加到顶点覆盖；(2) 删除 e 和与 v 相关的所有其他边。
 - 当然，这是一种输出顶点覆盖的近似算法。然而，可以证明该算法的性能比是无界的。（例如，星形图……）
- 但是，如果将边的两个端点都添加到顶点覆盖中，性能比将变为2。

11.4.3 最小顶点覆盖问题

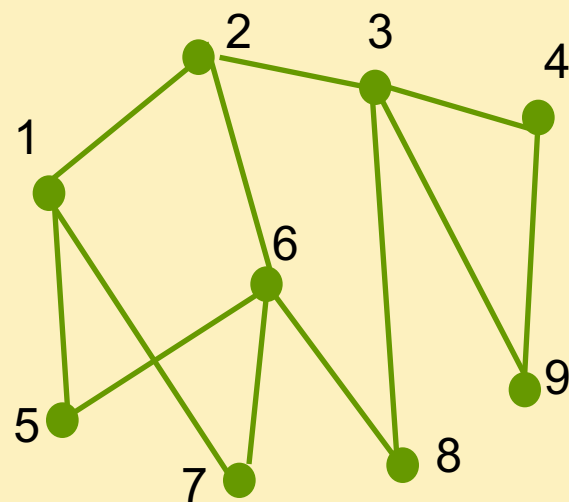
开始时令 $C=\emptyset$. 任取一条边 (u,v) , 把 u 和 v 加入 C 并删去 u 和 v 及其关联的边. 重复上述过程, 直至删去所有的边为止. C 为所求的顶点覆盖.



$\{1,2\}$

$\{1,2,3,4\}$

$\{1,2,3,4,5,6\}$



一个最优解: $\{1,3,6,9\}$, 近似解: $\{1,2,3,4,5,6\}$

MVC 算法

算法 11.2 MVC

输入: 无向图 $G=(V,E)$.

输出: G 的一个顶点覆盖 C .

1. $C \leftarrow \{\}$.
2. while $E \neq \{\}$
3. Let $e=(u,v)$ be any edge in E .
4. $C \leftarrow C \cup \{u,v\}$
5. 从 E 中删除 e 和所有与 u 或 v 相关联的边.
6. end while

- 时间复杂度 $O(m)$, $m=|E|$.

MVC算法的性能分析

- 算法第3步挑选的边是图的极大匹配 M . (M 是极大匹配, 表示没有其他匹配 M' , 使得 $M \subset M'$)
- 为了覆盖 M 中的边, 至少需要 $|M|$ 个顶点。
- 因此, 最小顶点覆盖的大小至少为 $|M|$ 。
- 但是MVC算法得到的覆盖大小为 $2|M|$ 。
- 因此, $R_{\text{MVC}} \leq 2$.

MVC算法的性能分析

该近似比可以达到。

例如：设图 G 由 k 条互不关联的边构成，显然

$$\text{MVC}(I) = 2k, \quad \text{OPT}(I) = k,$$

例如： $G = (\{v1, v2\}, \{(v1, v2)\})$ 。最优解 $\{v1\}$ ，而MVC算法得到的解 $\{v1, v2\}$ 。

结论：MVC 的近似比不会小于2，上面估计的MVC的近似比已不可能再进一步改进。MVC是2-近似算法。

11.4.4 多机调度问题-MPS

问题描述:

- 任给有穷的作业集 A 和 m 台相同的机器, 作业 a 的处理时间为正整数 $t(a)$, 每一项作业可以在任一台机器上处理. 如何把作业分配给机器才能使完成所有作业的时间最短? 即, 如何把 A 划分成 m 个不相交的子集 A_i 使得下式最小?

$$\max \left\{ \sum_{a \in A_i} t(a) \mid i = 1, 2, \dots, m \right\}$$

贪心法 (G-MPS)

- **负载**: 分配给一台机器的作业的处理时间之和.
- **贪心法 G-MPS**

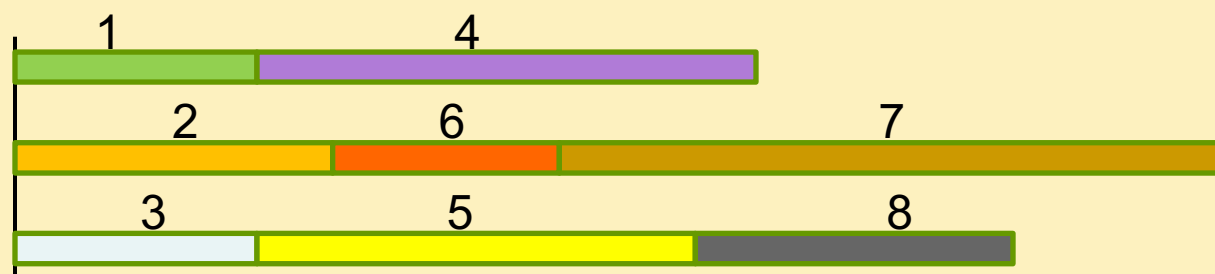
按输入的顺序分配作业;

把每一项作业分配给当前负载最小的机器 ;

如果当前负载最小的机器有 2 台或 2 台以上, 则分配给其中的任意一台 (比如标号最小的一台)

实例

例如，3 台机器，8 项作业，编号为1, 2, ... 8; 处理时间分别为 3, 4, 3, 6, 5, 3, 8, 4

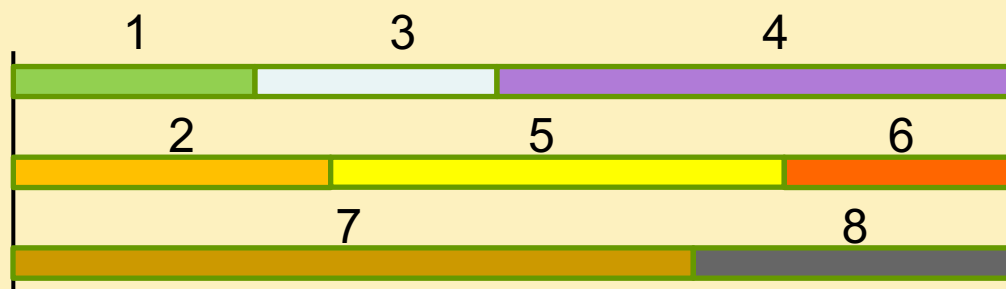


G-MPS的解
完成时间 15

算法给出的分配方案是 $\{1,4\}$, $\{2,6,7\}$, $\{3,5,8\}$,
负载分别为 $3+6=9$, $4+3+8=15$, $3+5+4=12$

实例

例如，3 台机器，8 项作业，编号为1, 2, ... 8; 处理时间分别为 3, 4, 3, 6, 5, 3, 8, 4



最优解
完成时间 12

最优的分配方案是 $\{1,3,4\}$, $\{2,5,6\}$, $\{7,8\}$,
负载分别为 $3+3+6=12$, $4+5+3=12$, $8+4=12$

G-MPS的性能分析

定理 对多机调度问题的每一个有 m 台机器的实例 I ,

$$\text{G-MPS}(I) \leq \left(2 - \frac{1}{m}\right) \text{OPT}(I).$$

证明: 显然有两个事实成立:

$$(1) \text{OPT}(I) \geq \frac{1}{m} \sum_{a \in A} t(a), (2) \text{OPT}(I) \geq \max_{a \in A} t(a).$$

(1) 表示 m 个机器的最大负载不小于平均负载

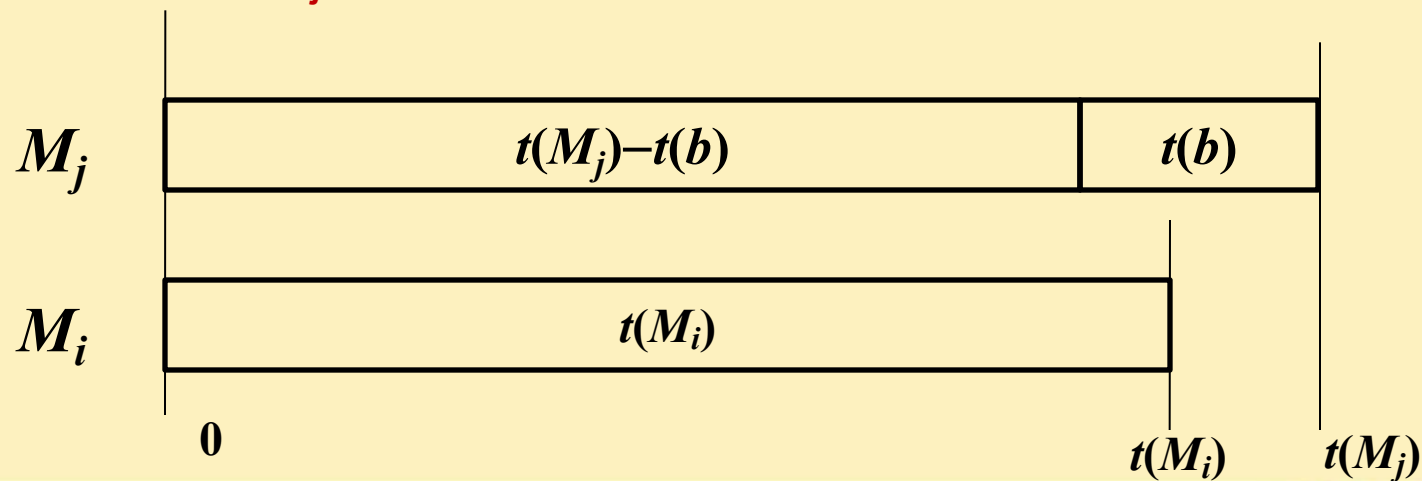
(2) 表示 m 个机器的最大负载不小于单个任务负载

G-MPS的性能分析

设机器 M_j 的负载最大，记作 $t(M_j)$ 。又设 b 是最后被分配给机器 M_j 的作业。根据算法，在考虑分配 b 时 M_j 的负载最小，故有

$$t(M_j) - t(b) \leq \frac{1}{m} \left(\sum_{a \in A} t(a) - t(b) \right).$$

分配 b 前 M_j 的负载不超过将除 b 外作业平均分配到 m 个机器的负载



G-MPS的性能分析

于是

$$\begin{aligned}\mathbf{G-MPS}(I) = t(M_j) &\leq \frac{1}{m} \left(\sum_{a \in A} t(a) - t(b) \right) + t(b) \\ &= \frac{1}{m} \sum_{a \in A} t(a) + \left(1 - \frac{1}{m} \right) t(b) \\ &\leq \mathbf{OPT}(I) + \left(1 - \frac{1}{m} \right) \mathbf{OPT}(I) \\ &= \left(2 - \frac{1}{m} \right) \mathbf{OPT}(I).\end{aligned}$$



紧实例

m 台机器, $m(m-1)+1$ 项作业,
前 $m(m-1)$ 项作业的处理时间都为 1, 最后一项作业的处理时间为 m .

算法方案: 把前 $m(m-1)$ 项作业平均地分配给 m 台机器, 每台 $m-1$ 项, 最后一项任意地分配给一台机器.

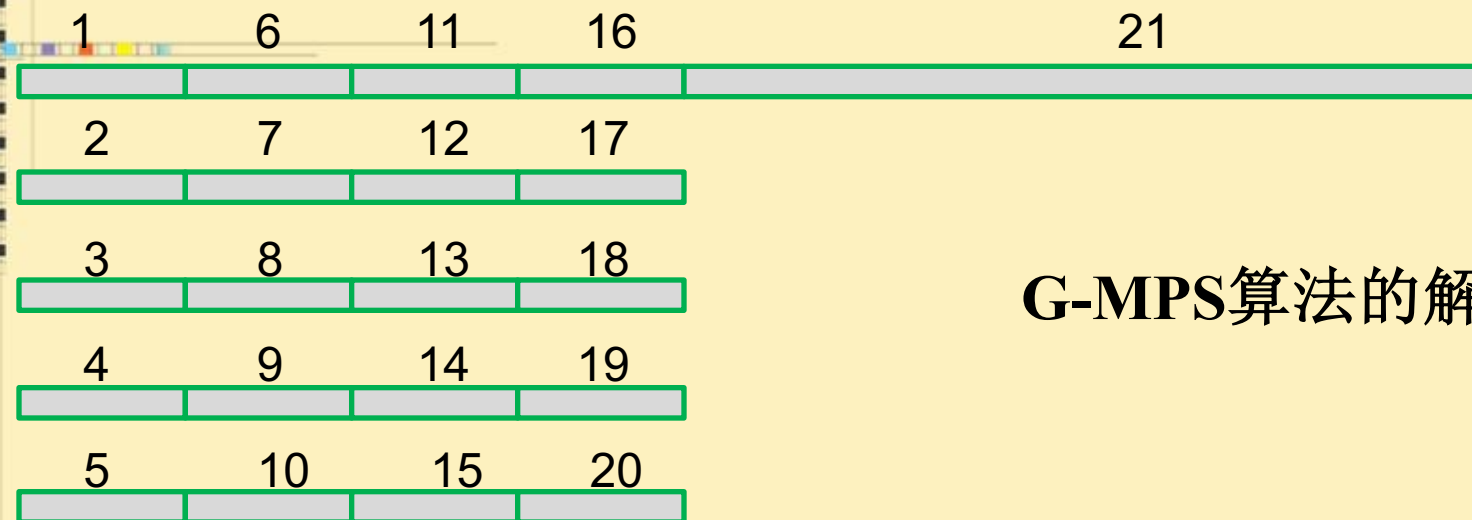
$$G\text{-MPS}(I) = 2m-1.$$

最优分配方案是把前 $m(m-1)$ 项作业平均地分配给 $m-1$ 台机器, 每台 m 项, 最后一项分配给留下的机器,

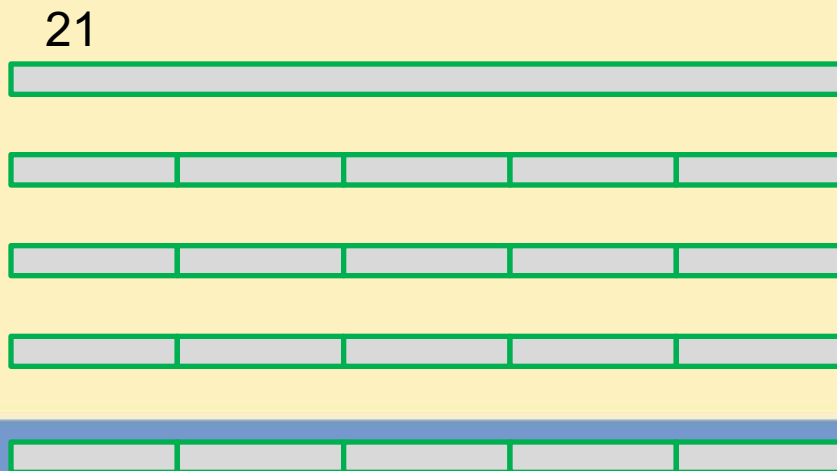
$$\text{OPT}(I) = m.$$

G-MPS 是 2-近似算法

例: $m=5$ 的紧实例



G-MPS算法的解



最优解

改进的贪心近似算法

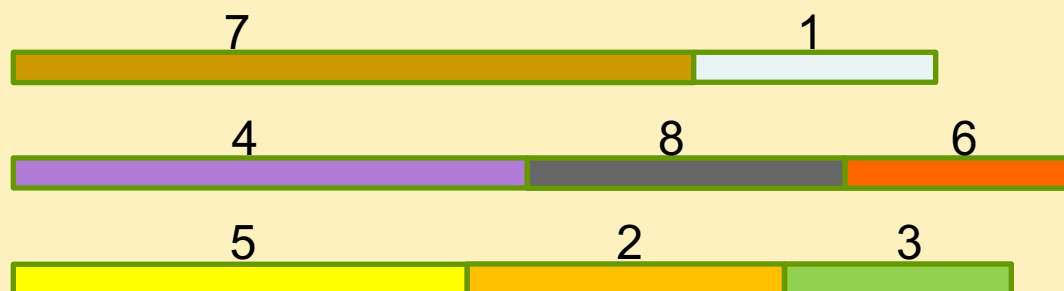
递降贪心法DG-MPS:

首先按处理时间从大到小重新排列作业（处理时间长的作业优先处理），然后运用G-MPS.

时间复杂度分析：DG-MPS增加排序时间 $O(n\log n)$, 仍然是多项式时间.

改进的贪心近似算法

- DG-MPS对上述紧实例得到最优解.
- 对之前的另一个实例的执行执行情况:
3 台机器, 作业编号(1,2,3,4,5,6,7,8), 处理时间 (3,4,3,6,5,3,8,4)
排序后:作业编号(7,4,5,2,8,1,3,6), 处理时间(8, 6, 5, 4, 4, 3, 3, 3)
3台机器的负载分别为 $8+3=11$, $6+4+3=13$, $5+4+3=12$.
比G-MPS的结果好.



**DG-MPS的解
完成时间13**

DG-MPS方法近似比分析

定理 对多机调度问题的每一个有 m 台机器的实例 I ,

$$\text{DG-MPS}(I) \leq \left(\frac{3}{2} - \frac{1}{2m}\right) \text{OPT}(I)$$

证明:

设作业按处理时间从大到小排列为 $a_1, a_2, \dots, a_m$, 仍考虑负载最大的机器 M_j 和最后分配给 M_j 的作业 a_i .

只有两种可能:

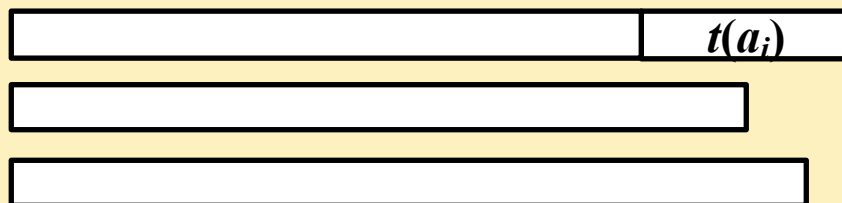
(1) M_j 只有一个作业, 则 $i=1$, 必为最优解.

DG-MPS方法近似比分析

(2) M_j 有 2 个或 2 个以上作业, 则 $i \geq m+1 \rightarrow n \geq m+1$ (每台机器至少已经分配 1 个作业) 则:

$$\text{OPT}(I) \geq t(a_m) + t(a_{m+1}) \geq 2 t(a_{m+1}) \geq 2 t(a_i)$$

M_j



作业 m 和 $m+1$ 必分配
在同一台机器

DG-MPS方法近似比分析

$$\begin{aligned}\text{DG-MPS}(I) &= t(M_j) \leq \frac{1}{m} \left(\sum_{k=1}^n t(a_k) - t(a_i) \right) + t(a_i) \\ &= \frac{1}{m} \sum_{k=1}^n t(a_k) + \left(1 - \frac{1}{m} \right) t(a_i) \leq \text{OPT}(I) + \left(1 - \frac{1}{m} \right) \cdot \frac{1}{2} \text{OPT}(I) \\ &= \left(\frac{3}{2} - \frac{1}{2m} \right) \text{OPT}(I)\end{aligned}$$



主要内容

11.1 引言

11.2 基本定义

11.3 近似算法分析

11.4 可近似问题算法举例

11.5 不可近似问题举例

11.6 完全可近似问题举例



11.4.5 不可近似问题举例

定理 货郎问题(不要求满足三角不等式)是不可近似的, 除非 $P = NP$.

证明: 假设不然, 设 A 是货郎问题的近似算法, 其近似比 $r \leq K$, K 是常数。

任给图 $G = \langle V, E \rangle$, 如下构造货郎问题的实例 I_G : 城市集 V , $\forall u, v \in V$,
若 $(u, v) \in E$, 则令 $d(u, v) = 1$; 否则令 $d(u, v) = Kn$, 其中 $|V| = n$.

若 G 有哈密顿回路, 则

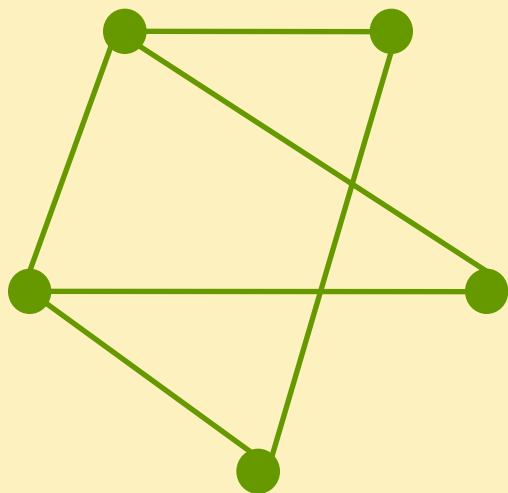
$$\text{OPT}(I_G) = n, \quad A(I_G) \leq r \text{OPT}(I_G) \leq Kn$$

否则

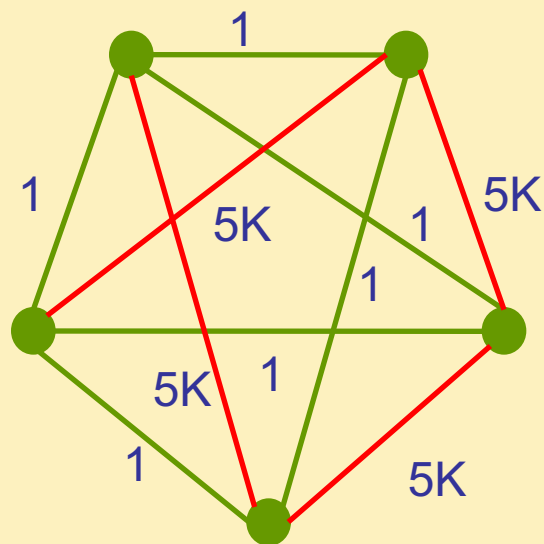
$$\text{OPT}(I_G) > Kn, \quad A(I_G) \geq \text{OPT}(I_G) > Kn$$

所以, G 有哈密顿回路当且仅当 $A(I_G) \leq Kn$

例



HC



货郎问题

HC实例有哈密顿回路，则货郎问题实例有长度不超过 n 的旅行，即近似算法的解不超过 Kn . 若HC实例没有哈密顿回路，则货郎实例的旅行路线长度大于 Kn .

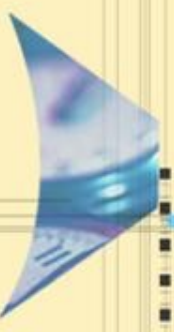
11.5 不可近似问题举例

(证明续) 下述算法可以判断图 G 是否有哈密顿回路.

算法

1. 由 I 构造货郎问题的实例 I_G
2. 对 I_G 运用近似算法 A
3. 若 $A(I_G) \leq Kn$, 则输出“Yes”; 否则输出“No”

由于 K 是固定的常数, 构造 I_G 可在 $O(n^2)$ 时间内完成且 $|I_G| = O(n^2)$. 由于 A 是多项式时间的, A 对 I_G 可在 n 的多项式时间内完成计算, 所以上述算法是 HC 的多项式时间算法。而 HC 是 NP 完全的, 推得 $P=NP$ (不太可能!).



主要内容

11.1 引言

11.2 基本定义

11.3 近似算法分析

11.4 可近似问题算法举例

11.5 不可近似问题举例

11.6 完全可近似问题举例



11.6 完全可近似问题举例

0/1 背包问题:

- 设 n 个物品 $\{u_1, u_2, \dots, u_n\}$ 要装进容量为 C 的背包中。对 $1 \leq j \leq n$, 令 w_j 和 v_j 分别为物品 u_j 的重量和价值。设所有物品的重量均不超过容量 C , 否则将物品排除在外。求最优的装包方案, 使得装进背包中的物品的总重量不超过背包容量, 且总价值最大。

11.6.1 简单的贪心算法

算法 G-KK

1. 按单位重量的价值从大到小排列物品. 设 $v_1/w_1 \geq v_2/w_2 \geq \dots \geq v_n/w_n$.
2. 顺序检查每一件物品, 只要能装得下就将它装入背包, 设装入背包的总价值为 V .
3. 求 $v_k = \max \{ v_i \mid i = 1, 2, \dots, n \}$.
若 $v_k > V$, 则将背包内的物品换成物品 u_k .

例

- 实例1: 四个物品(w, v): $u_1(3,7)$, $u_2(4,9)$, $u_3(5,8)$, $u_4(2,2)$; $C=6$.
- G-KK给出的解是装入 u_1 和 u_4 , 总价值为9.
- 实例2: 第3件物品 u_3 改为 $(5,10)$, 其他不变, 则G-KK 装入 u_3 , 总价值为10.
- 这两个实例的最优解都是装入 $u_2(4,9)$ 和 $u_4(2,2)$, 总价值为11.

G-KK算法的性能

定理：对0-1背包问题的任何实例 I , 有
$$\text{OPT}(I) < 2\text{G-KK}(I).$$

证明：

设物品 u_k 是第一件未装入背包的物品, 由于物品按单位重量的价值从大到小排列, 故有：

$$\begin{aligned}\text{OPT}(I) &< \text{G-KK}(I) + v_k \\ &\leq \text{G-KK}(I) + v_{\max} \quad (\text{最大价值}) \\ &\leq 2\text{G-KK}(I)\end{aligned}$$

11.6.2 多项式时间近似方案

算法 PTAS

输入: $k \geq 1$ 和实例 I .

1. 令 $\varepsilon = 1/k$.

2. 按单位重量的价值从大到小排列物品. 设

$$v_1/w_1 \geq v_2/w_2 \geq \dots \geq v_n/w_n.$$

3. 对每一个 $j=0,1,2,\dots,k$, 选出 j 件物品, 检查这 j 件物品的重量之和. 若它们的重量之和不超过 C , 则全部装进去, 接着用 G-KK 把剩余的物品装入背包.

4. 比较得到的所有装法, 取其中价值最大的作为近似解.

PTAS是一簇算法. 对每一个固定的 $k \geq 1$ ($\varepsilon = 1/k > 0$), PTAS 是一个算法, 记作 A_ε .

例子

取 $k=10$.

$j=0$: 直接使用 G-KK 算法

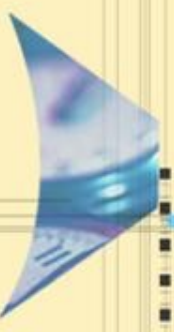
$j=1$: 尝试 n 次, 装入背包物品集分别为 $\{1\}, \{2\}, \dots, \{n\}$,
再使用 G-KK 算法

$j=2$: 尝试 $n(n-1)/2$ 次, 装入物品集分别是 $\{1, 2\}, \{1, 3\},$
 $\dots, \{n-1, n\}$, 再使用 G-KK 算法.

...

$j=10$: 尝试 C_n^{10} 次, 装入物品集为 $\{1, \dots, 9, 10\}, \{1, \dots, 9, 11\},$
 $\dots, \{n-9, n-8, \dots, n-1, n\}$, 再用 G-KK 算法.

总计运行 G-KK 算法次数: $\sum_{j=0}^k C_n^j$



PTAS的性能

定理： 令整数 $k \geq 1$, $\varepsilon = 1/k$ 。算法 A_ε 的运行时间为 $O(kn^{k+1})$, 近似度为 $1+\varepsilon$.

证明：

- (1) 算法的时间复杂度为 $O(kn^{k+1})$.
- (2) 算法的近似比的界为: $1+\varepsilon$.



(1)时间复杂度为 $O(kn^{k+1})$

- 由于 $\sum_{j=0}^k \binom{n}{j} = O(kn^k)$
- 因此大小不超过k的子集个数是 $O(kn^k)$
- 每次迭代的时间为 $O(n)$
- 因此，算法的时间复杂度为 $O(kn^{k+1})$

(2) 算法近似比的界为: $1+\varepsilon$

- 设0-1背包问题的实例为 I , 物品为 $U=\{u_1, u_2, \dots, u_n\}$, 背包容量为 C 。
- 设 X 是最优解对应的物品的集合。若 $|X| \leq k$, 则算法必得到 X , 因为算法尝试了所有大小不超过 k 的子集。因此假设 $|X| > k$ 。
 - 设 $Y=\{u_1, u_2, \dots, u_k\}$ 是 X 中 k 个具有最大价值的物品, (G-KK 算法以 Y 为基础计算近似解)。
 - 设 $Z=\{u_{k+1}, u_{k+2}, \dots, u_r\}$ 是最优解 X 中除 Y 外的余下物品, 并且假设 $v_j/s_j \geq v_{j+1}/s_{j+1}$, 对所有 j ($k+1 \leq j \leq r-1$)。
 - 因此最优解的值为, $OPT(I) = v_1 + v_2 + \dots + v_k + v_{k+1} + \dots + v_r$

(2) 算法近似比的界为: $1+\varepsilon$

- 既然Y中的物品是X中具有最大价值的物品, 因此有 $(k+1) v_j \leq \text{OPT}(I)$, $j=k+1, k+2, \dots, r$, 即有

$$v_j \leq \frac{\text{OPT}(I)}{k+1}, j = k+1, k+2, \dots, r$$

- 现考虑算法中的G-KK以Y为基础, 计算近似解。令 u_m 时Z中第一个没有被算法考虑装进背包的物品 (这样的物品必然存在, 否在算法输出了最优解)。
- 最优解的值可以写为:

$$\text{OPT}(I) = \sum_{j=1}^k v_j + \sum_{j=k+1}^{m-1} v_j + \sum_{j=m}^r v_j$$

(2) 算法近似比的界为: $1+\varepsilon$

- 令 W 表示算法在 u_m 之前装到背包中, 但又不在 $\{u_1, u_2, \dots, u_m\}$ 的物品集。换句话说讲, 如果 $u_j \in W$, 则 $u_j \notin \{u_1, u_2, \dots, u_m\}$, 并且 $v_j/s_j \geq v_m/s_m$ 。
- 因此, 算法得到的解的值可以写成:

$$A(I) = \sum_{j=1}^k v_j + \sum_{j=k+1}^{m-1} v_j + \sum_{u_j \in W} v_j + \text{items values after } u_m$$

- 因此有

$$A(I) \geq \sum_{j=1}^k v_j + \sum_{j=k+1}^{m-1} v_j + \sum_{u_j \in W} v_j$$

(2) 算法近似比的界为: $1+\varepsilon$

- 对U-Y 中 u_{m-1} 之后的物品, 令

$$C' = C - \sum_{j=1}^k s_j - \sum_{j=k+1}^{m-1} s_j, C'' = C' - \sum_{u_j \in W} s_j$$

- 显然, C' 对应最优解中在考虑 u_m 时背包的剩余容量;
 C'' (为近似解中在考虑 u_m 时背包的剩余容量。
- 因此有

$$OPT(I) \leq \sum_{j=1}^k v_j + \sum_{j=k+1}^{m-1} v_j + C' \frac{v_m}{s_m}$$

Z 中, $v_j/s_j \geq v_{j+1}/s_{j+1}$ 对所有 $j, k+1 \leq j \leq r-1$

(2) 算法近似比的界为: $1+\varepsilon$

由于算法装不进最优解中存在的物品 u_m , 因此一定有 $C'' < s_m$

另外, W 是算法在 u_m 之前装到背包中的物品集, 因此有 $v_j/s_j \geq v_m/s_m$, 对每个 W 中的物品 u_j 。

这样, 一定有

$$OPT(I) < \sum_{j=1}^k v_j + \sum_{j=k+1}^{m-1} v_j + \sum_{u_j \in W} v_j + v_m < A(I) + \frac{OPT(I)}{k+1}$$

$$R_k = \frac{OPT(I)}{A(I)} < 1 + \frac{1}{k} = 1 + \varepsilon$$

11.6.3 完全多项式近似方案

- 上述描述的多项式近似方案PTAS的运行时间对于 $1/\varepsilon$ 是以指数形式增长的， $1/\varepsilon$ 是预期的误差界的倒数。
- 而完全多项式近似方案，运行时间同时也是 $1/\varepsilon$ 的多项式的。
- 对于一些NP难的问题，可以用约束的近似方案得到。
 - 对于存在伪多项式时间算法A的NP困难问题，找一个FPAS的一般思路：从A开始，对于实例I的输入值应用标度变换和舍入得到实例I'，将算法A应用到修改得到的实例I'上得出一个答案，它就是相对于实例I的最优解的近似解。

算法4.8 KNAPSACK

输入: 物品集合 $U = \{u_1, u_2, \dots, u_n\}$, 体积分别为 $s_1, s_2, \dots, s_n$, 价值分别为 $v_1, v_2, \dots, v_n$, 容量为 C 的背包。

输出: $\sum_{u_i \in S} v_i$ 的最大总价值, 且满足 $\sum_{u_i \in S} s_i \leq C$, 其中 $S \subseteq U$.

```
1. for i ← 0 to n
2.      $V[i, 0] \leftarrow 0$ 
3. end for
4. for j ← 0 to n
5.      $V[0, j] \leftarrow 0$ 
6. end for
7. for i ← 1 to n
8.     for j ← 1 to C
9.          $V[i, j] \leftarrow V[i-1, j]$ 
10.        if  $s_i \leq j$  then  $V[i, j] \leftarrow \max\{V[i, j], V[i-1, j-s_i] + v_i\}$ 
11.    end for
12. end for
13. return  $V[n, C]$ 
```

构造完全多项式时间近似方案

算法 FPTAS

输入: 整数 $k > 0$ 和实例 I .

1. 令 $K = \frac{C}{2(k+1)n}$
2. 令 $C' = \lfloor C/K \rfloor$, $s_j' = \lfloor s_j/K \rfloor$, $v_j' = \lfloor v_j/K \rfloor$, $1 \leq i \leq n$, 记新实例为 I' .
3. 对 I' 应用算法 KNAPSACK 得到解 S , 把 S 取作实例 I 的解.

FPTAS 为一个算法簇。对于每个 k , $\varepsilon = 1/k$, 每个算法为近似算法, 记为 A_ε

A_ε 性能分析

定理：对每一个整数 $k > 1$, $\varepsilon = 1/k$, 设 0-1 背包问题的实例 I , 有: $\text{OPT}(I) < (1 + \varepsilon) A_\varepsilon(I)$, 并且 A_ε 的时间复杂度为 $\Theta(n^2/\varepsilon)$ 。

证明：(1) 算法的运行时间 $\Theta(nC/K) = \Theta(kn^2) = \Theta(n^2/\varepsilon)$ 。

(2) 近似度分析:

- 由于最优解最多包含 n 个物品, 因此有

$$\text{OPT}(I) - K * \text{OPT}(I') \leq Kn$$

- 对于实例 I 的近似解的值是算法对于实例 I' 输出的最优解值的 K 倍, 即有: $\text{OPT}(I) - A_\varepsilon(I) \leq Kn$
- 因此, $A_\varepsilon(I) \geq \text{OPT}(I) - Kn = \text{OPT}(I) - C/(2k+2)$
- 不失一般性, 假设 $\text{OPT}(I) \geq C/2$ (否则很容易得到最优解), 得到 $R_{A_\varepsilon}(I) \leq 1 + 1/k = 1 + \varepsilon$.